

JAVA CODING CHALLENGE

Insurance Management System

ENTITY PACKAGES:

1.Client Class

```
package entity;
```

```
public class Client {
```

```
    private String clientId;
```

```
    private String clientName;
```

```
    private String contactInfo;
```

```
    private Policy policy;
```

```
    public Client() {}
```

```
    public Client(String clientId, String clientName, String contactInfo,  
Policy policy) {
```

```
        this.clientId = clientId;
```

```
        this.clientName = clientName;
```

```
        this.contactInfo = contactInfo;
```

```
        this.policy = policy;
```

```
    }
```

```
    public String getClientId() { return clientId; }
```

```
public void setClientId(String clientId) { this.clientId = clientId; }  
public String getClientName() { return clientName; }  
public void setClientName(String clientName) { this.clientName =  
clientName; }  
public String getContactInfo() { return contactInfo; }  
public void setContactInfo(String contactInfo) { this.contactInfo =  
contactInfo; }  
public Policy getPolicy() { return policy; }  
public void setPolicy(Policy policy) { this.policy = policy; }
```

@Override

```
public String toString() {  
    return "Client{clientId=" + clientId + ", clientName=" +  
clientName + ", contactInfo=" + contactInfo + ", policy=" + policy + "}";  
}  
}
```

2.User Class

package entity;

```
public class User {  
    private String userId;  
    private String username;  
    private String password;  
    private String role;  
  
    // Default Constructor  
    public User() {}
```

// Parameterized Constructor

```
public User(String userId, String username, String password, String
role) {
    this.userId = userId;
    this.username = username;
    this.password = password;
    this.role = role;
}
```

// Getters and Setters

```
public String getUserId() { return userId; }
public void setUserId(String userId) { this.userId = userId; }

public String getUsername() { return username; }
public void setUsername(String username) { this.username =
username; }

public String getPassword() { return password; }
public void setPassword(String password) { this.password = password;
}

public String getRole() { return role; }
public void setRole(String role) { this.role = role; }
```

// toString() Method

@Override

```
public String toString() {  
    return "User{userId=\"" + userId + "\", username=\"" + username + "\",  
role=\"" + role + "\"}";  
}  
}
```

3. Policy Class

package entity;

```
public class Policy {  
    private String policyId;  
    private String policyName;  
    private double coverageAmount;  
    private double premium;  
  
    // Default Constructor  
    public Policy() {}  
  
    // Parameterized Constructor  
    public Policy(String policyId, String policyName, double  
coverageAmount, double premium) {  
        this.policyId = policyId;  
        this.policyName = policyName;  
        this.coverageAmount = coverageAmount;  
        this.premium = premium;  
    }  
}
```

```
// Getters and Setters
```

```
public String getPolicyId() { return policyId; }
```

```
public void setPolicyId(String policyId) { this.policyId = policyId; }
```

```
public String getPolicyName() { return policyName; }
```

```
public void setPolicyName(String policyName) { this.policyName =  
policyName; }
```

```
public double getCoverageAmount() { return coverageAmount; }
```

```
public void setCoverageAmount(double coverageAmount) {  
this.coverageAmount = coverageAmount; }
```

```
public double getPremium() { return premium; }
```

```
public void setPremium(double premium) { this.premium = premium;  
}
```

```
// toString() Method
```

```
@Override
```

```
public String toString() {
```

```
    return "Policy{" +
```

```
        "policyId=" + policyId + "\n" +
```

```
        ", policyName=" + policyName + "\n" +
```

```
        ", coverageAmount=" + coverageAmount +
```

```
        ", premium=" + premium +
```

```
        "}";
```

```
}
```

```
}
```

4.Claim Class

```
package entity;
```

```
import java.util.Date;
```

```
public class Claim {
```

```
    private int claimId;
```

```
    private String claimNumber;
```

```
    private Date dateFiled;
```

```
    private double claimAmount;
```

```
    private String status;
```

```
    private Policy policy; // Many-to-One Relationship
```

```
    private Client client; // Many-to-One Relationship
```

```
    // Default Constructor
```

```
    public Claim() {}
```

```
    // Parameterized Constructor
```

```
    public Claim(int claimId, String claimNumber, Date dateFiled, double  
claimAmount, String status, Policy policy, Client client) {
```

```
        this.claimId = claimId;
```

```
        this.claimNumber = claimNumber;
```

```
        this.dateFiled = dateFiled;
```

```
        this.claimAmount = claimAmount;
```

```
        this.status = status;
```

```
        this.policy = policy;
```

```
        this.client = client;
    }

    // Getters and Setters
    public int getClaimId() {
        return claimId;
    }

    public void setClaimId(int claimId) {
        this.claimId = claimId;
    }

    public String getClaimNumber() {
        return claimNumber;
    }

    public void setClaimNumber(String claimNumber) {
        this.claimNumber = claimNumber;
    }

    public Date getDateFiled() {
        return dateFiled;
    }

    public void setDateFiled(Date dateFiled) {
        this.dateFiled = dateFiled;
    }
}
```

```
}
```

```
public double getClaimAmount() {  
    return claimAmount;  
}
```

```
public void setClaimAmount(double claimAmount) {  
    this.claimAmount = claimAmount;  
}
```

```
public String getStatus() {  
    return status;  
}
```

```
public void setStatus(String status) {  
    this.status = status;  
}
```

```
public Policy getPolicy() {  
    return policy;  
}
```

```
public void setPolicy(Policy policy) {  
    this.policy = policy;  
}
```

```
public Client getClient() {  
    return client;  
}
```

```
public void setClient(Client client) {  
    this.client = client;  
}
```

```
// toString() Method for Displaying Claim Details
```

```
@Override
```

```
public String toString() {  
    return "Claim{" +  
        "claimId=" + claimId +  
        ", claimNumber=" + claimNumber + "\" +  
        ", dateFiled=" + dateFiled +  
        ", claimAmount=" + claimAmount +  
        ", status=" + status + "\" +  
        ", policy=" + (policy != null ? policy.getPolicyId() : "null") +  
        ", client=" + (client != null ? client.getClientId() : "null") +  
        "}";  
}  
}
```

5.Payment Class

```
package entity;
```

```
import java.util.Date;
```

```
public class Payment {  
    private int paymentId;  
    private Date paymentDate;  
    private double paymentAmount;  
    private Client client; // Many-to-One Relationship  
  
    // Default Constructor  
    public Payment() {}  
  
    // Parameterized Constructor  
    public Payment(int paymentId, Date paymentDate, double  
paymentAmount, Client client) {  
        this.paymentId = paymentId;  
        this.paymentDate = paymentDate;  
        this.paymentAmount = paymentAmount;  
        this.client = client;  
    }  
  
    // Getters and Setters  
    public int getPaymentId() {  
        return paymentId;  
    }  
  
    public void setPaymentId(int paymentId) {  
        this.paymentId = paymentId;  
    }  
}
```

```
}
```

```
public Date getPaymentDate() {  
    return paymentDate;  
}
```

```
public void setPaymentDate(Date paymentDate) {  
    this.paymentDate = paymentDate;  
}
```

```
public double getPaymentAmount() {  
    return paymentAmount;  
}
```

```
public void setPaymentAmount(double paymentAmount) {  
    this.paymentAmount = paymentAmount;  
}
```

```
public Client getClient() {  
    return client;  
}
```

```
public void setClient(Client client) {  
    this.client = client;  
}
```

```

// toString() Method for Displaying Payment Details
@Override
public String toString() {
    return "Payment{" +
        "paymentId=" + paymentId +
        ", paymentDate=" + paymentDate +
        ", paymentAmount=" + paymentAmount +
        ", client=" + (client != null ? client.getClientId() : "null") +
        '}';
}
}

```

DAO PACKAGES:

1. IPolicyService Interface

```
package dao;
```

```
import entity.Policy;
```

```
import java.util.Collection;
```

```
import myexceptions.PolicyNotFoundException;
```

```
public interface IPolicyService {
```

```
    // Method to create a new policy
```

```
    boolean createPolicy(Policy policy);
```

```
    // Method to retrieve a policy by its ID
```

```
    Policy getPolicy(String searchId) throws PolicyNotFoundException;
```

```

// Method to get all policies
Collection<Policy> getAllPolicies();

// Method to update an existing policy
boolean updatePolicy(Policy policy);

// Method to delete a policy by its ID
boolean deletePolicy(String policyId);
}

2. InsuranceServiceImpl
package dao;

import entity.Policy;
import java.sql.Connection;
import java.sql.PreparedStatement;
import java.sql.ResultSet;
import java.sql.SQLException;
import java.sql.Statement;
import java.util.ArrayList;
import java.util.Collection;
import myexceptions.PolicyNotFoundException;
import util.DBConnection;

public class InsuranceServiceImpl implements IPolicyService {

```

```

// Create a new policy and insert it into the database

@Override
public boolean createPolicy(Policy policy) {
    String query = "INSERT INTO policy (policyId, policyName,
coverageAmount, premium) VALUES (?, ?, ?, ?)";

    try (Connection conn = DBConnection.getConnection();
        PreparedStatement pstmt = conn.prepareStatement(query)) {

        pstmt.setString(1, policy.getPolicyId());
        pstmt.setString(2, policy.getPolicyName());
        pstmt.setDouble(3, policy.getCoverageAmount());
        pstmt.setDouble(4, policy.getPremium());

        return pstmt.executeUpdate() > 0; // Returns true if insertion is
successful

    } catch (SQLException e) {
        e.printStackTrace();
    }
    return false;
}

// Retrieve a policy by ID

@Override
public Policy getPolicy(String policyId) throws
PolicyNotFoundException {
    String query = "SELECT * FROM policy WHERE policyId = ?";

```

```

try (Connection conn = DBConnection.getConnection();
    PreparedStatement stmt = conn.prepareStatement(query)) {

    stmt.setString(1, policyId);

    try (ResultSet rs = stmt.executeQuery()) {
        if (rs.next()) {
            return new Policy(
                rs.getString("policyId"),
                rs.getString("policyName"),
                rs.getDouble("coverageAmount"),
                rs.getDouble("premium")
            );
        } else {
            throw new PolicyNotFoundException("Policy with ID " +
policyId + " not found.");
        }
    }
} catch (SQLException e) {
    e.printStackTrace();
}

return null;
}

```

// Retrieve all policies

@Override

```

public Collection<Policy> getAllPolicies() {
    String query = "SELECT * FROM policy";
    Collection<Policy> policies = new ArrayList<>();

    try (Connection conn = DBConnection.getConnection();
        Statement stmt = conn.createStatement();
        ResultSet rs = stmt.executeQuery(query)) {

        while (rs.next()) {
            policies.add(new Policy(
                rs.getString("policyId"),
                rs.getString("policyName"),
                rs.getDouble("coverageAmount"),
                rs.getDouble("premium")
            ));
        }

    } catch (SQLException e) {
        e.printStackTrace();
    }

    return policies;
}

// Update an existing policy
@Override
public boolean updatePolicy(Policy policy) {

```

```
String query = "UPDATE policy SET policyName = ?,  
coverageAmount = ?, premium = ? WHERE policyId = ?";
```

```
try (Connection conn = DBConnection.getConnection();  
    PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
    pstmt.setString(1, policy.getPolicyName());
```

```
    pstmt.setDouble(2, policy.getCoverageAmount());
```

```
    pstmt.setDouble(3, policy.getPremium());
```

```
    pstmt.setString(4, policy.getPolicyId());
```

```
    return pstmt.executeUpdate() > 0; // Returns true if update is  
    successful
```

```
    } catch (SQLException e) {
```

```
        e.printStackTrace();
```

```
    }
```

```
    return false;
```

```
}
```

```
// Delete a policy by ID
```

```
@Override
```

```
public boolean deletePolicy(String policyId) {
```

```
    String query = "DELETE FROM policy WHERE policyId = ?";
```

```
    try (Connection conn = DBConnection.getConnection();
```

```
        PreparedStatement pstmt = conn.prepareStatement(query)) {
```

```
        pstmt.setString(1, policyId);  
        return pstmt.executeUpdate() > 0; // Returns true if deletion is  
successful
```

```
    } catch (SQLException e) {  
        e.printStackTrace();  
    }  
    return false;  
}  
}
```

MY EXCEPTION PACKAGE:

1. PolicyNotFoundException Class

```
package myexceptions;
```

```
public class PolicyNotFoundException extends Exception {  
    // Constructor with error message  
    public PolicyNotFoundException(String message) {  
        super(message);  
    }  
}
```

UTIL PACKAGE:

1.DBConnection Class

```
package util;
```

```
import java.sql.Connection;  
import java.sql.DriverManager;
```

```
import java.sql.SQLException;
import java.util.Properties;

public class DBConnection {

    // Method to get a new connection each time
    public static Connection getConnection() {
        try {
            // Load database properties
            Properties properties = Propertyutil.loadProperties();
            if (properties == null) {
                System.err.println(" Failed to load database properties.");
                return null;
            }

            // Extract database properties
            String dbUrl = properties.getProperty("db.url");
            String dbUser = properties.getProperty("db.username");
            String dbPassword = properties.getProperty("db.password");

            // Ensure properties are not null
            if (dbUrl == null || dbUser == null || dbPassword == null) {
                System.err.println(" Database properties are missing!");
                return null;
            }
        }
    }
}
```

```

        Connection conn = DriverManager.getConnection(dbUrl, dbUser,
dbPassword);

        System.out.println(" Database connected successfully!");

        return conn;

    } catch (SQLException e) {

        System.err.println(" Database connection error: " +
e.getMessage());

        e.printStackTrace();

        return null;

    }

}
}
}

```

2. Propertyutil Class

```

package util;

import java.io.IOException;
import java.io.InputStream;
import java.util.Properties;

public class Propertyutil {

    public static Properties loadProperties() {

        Properties properties = new Properties();

        try                (InputStream                input                =
Propertyutil.class.getClassLoader().getResourceAsStream("dbconfig.pro
perties")) {

            if (input == null) {

```

```

        System.err.println("Unable to find dbconfig.properties in
resources.");
        return null;
    }
    properties.load(input);
} catch (IOException e) {
    System.err.println("Failed to load database properties.");
    e.printStackTrace();
}
return properties;
}

```

```

public static String getPropertyString() {
    Properties properties = loadProperties();
    if (properties == null) return null;

    return properties.getProperty("db.url") + "?user=" +
properties.getProperty("db.username") +
        "&password=" + properties.getProperty("db.password");
}
}

```

MAINMOD PACKAGE

1. MainModule

```

package mainmod;
import dao.IPolicyService;
import dao.InsuranceServiceImpl;

```

```
import entity.Policy;
import java.util.Scanner;
import myexceptions.PolicyNotFoundException;

public class MainModule {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        IPolicyService policyService = new InsuranceServiceImpl();

        while (true) {
            System.out.println("\nInsurance Management System");
            System.out.println("1.Create Policy");
            System.out.println("2.Get Policy by ID");
            System.out.println("3.Get All Policies");
            System.out.println("4.Update Policy");
            System.out.println("5.Delete Policy");
            System.out.println("6.Exit");
            System.out.print("Enter your choice: ");

            int choice = scanner.nextInt();
            scanner.nextLine(); // Consume newline

            switch (choice) {
                case 1:
                    System.out.print("Enter Policy ID: ");
                    String policyId = scanner.next();
```

```
scanner.nextLine(); // Consume newline
```

```
System.out.print("Enter Policy Name: ");
```

```
String policyName = scanner.nextLine();
```

```
System.out.print("Enter Coverage Amount: ");
```

```
double coverageAmount = scanner.nextDouble();
```

```
System.out.print("Enter Premium Amount: ");
```

```
double premium = scanner.nextDouble();
```

```
scanner.nextLine(); // Consume newline
```

```
Policy policy = new Policy(policyId, policyName,  
coverageAmount, premium);
```

```
System.out.println("Trying to insert Policy ID:" + policyId);
```

```
boolean created = policyService.createPolicy(policy);
```

```
System.out.println("Policy creation result: " + created);
```

```
System.out.println(created ? "Policy created successfully..." :  
"Policy already Exists.");
```

```
break;
```

```
case 2:
```

```
System.out.print("Enter Policy ID to fetch: ");
```

```
String searchId = scanner.next();
```

```
scanner.nextLine(); // Consume newline
```

```
try {  
    Policy fetchedPolicy = policyService.getPolicy(searchId);  
    System.out.println("Policy Found: " + fetchedPolicy);  
} catch (PolicyNotFoundException e) {  
    System.out.println(e.getMessage());  
}  
break;
```

case 3:

```
System.out.println("Displaying all policies from the  
Database");  
for (Policy p : policyService.getAllPolicies()) {  
    System.out.println(p);  
}  
break;
```

case 4:

```
System.out.print("Enter Policy ID to update: ");  
String updateId = scanner.next();  
scanner.nextLine(); // Consume newline  
  
System.out.print("Enter New Policy Name: ");  
String newName = scanner.nextLine();  
  
System.out.print("Enter New Coverage Amount: ");  
double newCoverage = scanner.nextDouble();
```

```
System.out.print("Enter New Premium Amount: ");  
double newPremium = scanner.nextDouble();  
scanner.nextLine(); // Consume newline
```

```
Policy updatedPolicy = new Policy(updateId, newName,  
newCoverage, newPremium);  
boolean updated =  
policyService.updatePolicy(updatedPolicy);  
System.out.println(updated ? "Policy updated successfully..."  
: "Such Policy Not Found.");  
break;
```

case 5:

```
System.out.print("Enter Policy ID to delete: ");  
String deleteId = scanner.next();  
scanner.nextLine(); // Consume newline
```

```
boolean deleted = policyService.deletePolicy(deleteId);  
System.out.println(deleted ? "Policy deleted successfully..." :  
"Such Policy Not Found.");  
break;
```

case 6:

```
System.out.println("Exiting... Thank you!");  
scanner.close();  
System.exit(0);
```

```
        break;

    default:
        System.out.println("Invalid choice, Please try again.");
    }
}
}
```

DATABASE PROPERTIES:

dbconfig.properties

db.url=jdbc:mysql://localhost:3306/database_name

db.username=root

db.password=your_password

OUTPUTS:

```
Insurance Management System
1.Create Policy
2.Get Policy by ID
3.Get All Policies
4.Update Policy
5.Delete Policy
6.Exit
Enter your choice: 1
Enter Policy ID: P59
Enter Policy Name: Bike Secure
Enter Coverage Amount: 500000
Enter Premium Amount: 3000
Trying to insert Policy ID:P59
Database connected successfully!
Policy creation result: true
Policy created successfully...
```

```
Insurance Management System
1.Create Policy
2.Get Policy by ID
3.Get All Policies
4.Update Policy
5.Delete Policy
6.Exit
Enter your choice: 2
Enter Policy ID to fetch: P59
Database connected successfully!
Policy Found: Policy{policyId='P59', policyName='Bike Secure', coverageAmount=500000.0, premium=3000.0}
```

```
Insurance Management System
1.Create Policy
2.Get Policy by ID
3.Get All Policies
4.Update Policy
5.Delete Policy
6.Exit
Enter your choice: 3
Displaying all policies from the Database
Database connected successfully!
Policy{policyId='P10', policyName='Family Health Plan', coverageAmount=600000.0, premium=2000.0}
Policy{policyId='P30', policyName='Travel Safe', coverageAmount=300000.0, premium=500.0}
Policy{policyId='P42', policyName='Car Secure', coverageAmount=700000.0, premium=4000.0}
Policy{policyId='P50', policyName='Health Plus', coverageAmount=2000000.0, premium=10000.0}
Policy{policyId='P59', policyName='Bike Secure', coverageAmount=500000.0, premium=3000.0}
```

```
Insurance Management System
1.Create Policy
2.Get Policy by ID
3.Get All Policies
4.Update Policy
5.Delete Policy
6.Exit
Enter your choice: 4
Enter Policy ID to update: P59
Enter New Policy Name: Bike Secure
Enter New Coverage Amount: 320000
Enter New Premium Amount: 1000
Database connected successfully!
Policy updated successfully...
```

Insurance Management System

- 1.Create Policy
- 2.Get Policy by ID
- 3.Get All Policies
- 4.Update Policy
- 5.Delete Policy
- 6.Exit

Enter your choice: 5

Enter Policy ID to delete: P42

Database connected successfully!

Policy deleted successfully...

Insurance Management System

- 1.Create Policy
- 2.Get Policy by ID
- 3.Get All Policies
- 4.Update Policy
- 5.Delete Policy
- 6.Exit

Enter your choice: 6

Exiting... Thank you!

